Interprocess Communication - Part 2

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 POSIX Message Queues

Message queues are another IPC mechanism. In contrast to pipes and FIFOs, they are *message-oriented*: boundaries between messages are preserved. Also, each message has a priority, and higher-priority messages are received first.

For POSIX message queues, useful manual pages are:

```
man 7 mq_overview
man 3 mq_open
man 3 mq_send
man 3 mq_receive
```

1.1 Basic queue operations and priorities

The following program creates a queue, sends three messages with different priorities, then receives them again:

```c
#include <errno.h>
#include <mqueue.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define QUEUE_NAME "/mq_demo_basic"
#define MSG_SIZE 128

int main(void) {
    struct mq_attr attr = {0};
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = MSG_SIZE;

    mq_unlink(QUEUE_NAME);

    mqd_t mq = mq_open(QUEUE_NAME, O_CREAT | O_RDWR, 0600, &attr);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        return EXIT_FAILURE;
    }

    if (mq_send(mq, "low priority", strlen("low priority") + 1, 1) == -1) {
        perror("mq_send low");
        mq_close(mq);
        mq_unlink(QUEUE_NAME);
        return EXIT_FAILURE;
    }

    if (mq_send(mq, "high priority", strlen("high priority") + 1, 9) == -1) {
        perror("mq_send high");
        mq_close(mq);
        mq_unlink(QUEUE_NAME);
        return EXIT_FAILURE;
```

```c
    }

    if (mq_send(mq, "medium priority", strlen("medium priority") + 1, 5) == -1) {
        perror("mq_send medium");
        mq_close(mq);
        mq_unlink(QUEUE_NAME);
        return EXIT_FAILURE;
    }

    for (int i = 0; i < 3; i++) {
        char buf[MSG_SIZE];
        unsigned int prio = 0;
        ssize_t n = mq_receive(mq, buf, sizeof(buf), &prio);
        if (n == -1) {
            perror("mq_receive");
            mq_close(mq);
            mq_unlink(QUEUE_NAME);
            return EXIT_FAILURE;
        }
        printf("received (prio %u): %s\n", prio, buf);
    }

    if (mq_close(mq) == -1) {
        perror("mq_close");
        mq_unlink(QUEUE_NAME);
        return EXIT_FAILURE;
    }

    if (mq_unlink(QUEUE_NAME) == -1) {
        perror("mq_unlink");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Important details:

- `mq_unlink(name)` removes the queue *name* from the system namespace. Existing open descriptors remain valid, but no new process can open that queue by name anymore. The queue object is fully deleted only after the last open descriptor is closed.
- POSIX queue names must start with `/`, for example `/mq_demo_basic`. They are not normal filesystem paths and should not contain additional slashes.
- `mq_open(name, oflag, mode, attr)`:
  - `name`: queue identifier (for example `/mq_demo_basic`)
  - `oflag`: access and creation flags (`O_CREAT`, `O_RDONLY`, `O_WRONLY`, `O_RDWR`, ...)
  - `mode`: permission bits used when `O_CREAT` creates a new queue

  - `attr`: queue attributes used on creation (`mq_maxmsg`, `mq_msgsize`)

- `mq_send(mqdes, msg_ptr, msg_len, msg_prio)`:
  - `mqdes`: The descriptor of the message queue to send the message to.
  - `msq_ptr`: The address of the data to send.
  - `msq_len`: The number of bytes of the data pointed to by `msq_ptr`.
  - `msq_prio`: The priority of the message (high values correspond to high priority).

- `mq_close(mqdes)` closes the message queue descriptor in this process, and the kernel removes the queue descriptor from this process's descriptor table. But it does not delete the queue or remove its name from the system namespace.

Run:

```
./06_mqueue_basic
```

output:

```
received (prio 9): high priority
received (prio 5): medium priority
received (prio 1): low priority
```

Although the messages were sent in the order low, high, medium, they are received in priority order. This is a core behavior of POSIX message queues.

1.2 Two-process communication with a named queue

Now we use two separate programs. One acts as receiver and creates the queue. The other acts as sender and opens the existing queue for writing.

Receiver

```c
#include <mqueue.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define QUEUE_NAME "/mq_demo_ipc"
#define MSG_SIZE 128

int main(void) {
    struct mq_attr attr = {0};
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = MSG_SIZE;
    pid_t pid = getpid();

    mq_unlink(QUEUE_NAME);

    mqd_t mq = mq_open(QUEUE_NAME, O_CREAT | O_RDONLY, 0600, &attr);
```

```c
    if (mq == (mqd_t)-1) {
        perror("mq_open receiver");
        return EXIT_FAILURE;
    }

    printf("receiver %d: queue created (%s)\n", pid, QUEUE_NAME);
    printf("receiver %d: waiting for sender...\n", pid);

    for (int i = 0; i < 3; i++) {
        sleep(3);
        char buf[MSG_SIZE];
        unsigned int prio = 0;
        ssize_t n = mq_receive(mq, buf, sizeof(buf), &prio);
        if (n == -1) {
            perror("mq_receive");
            mq_close(mq);
            mq_unlink(QUEUE_NAME);
            return EXIT_FAILURE;
        }
        printf("receiver %d: got (prio %u): %s\n", pid, prio, buf);
    }

    if (mq_close(mq) == -1) {
        perror("mq_close receiver");
        mq_unlink(QUEUE_NAME);
        return EXIT_FAILURE;
    }

    if (mq_unlink(QUEUE_NAME) == -1) {
        perror("mq_unlink receiver");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Sender

```c
#include <mqueue.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define QUEUE_NAME "/mq_demo_ipc"

int main(void) {
    mqd_t mq = mq_open(QUEUE_NAME, O_WRONLY);
    if (mq == (mqd_t)-1) {
        perror("mq_open sender");
        return EXIT_FAILURE;
    }

```

```
    if (mq_send(mq, "message with prio 1",
                strlen("message with prio 1") + 1, 1) == -1) {
        perror("mq_send prio1");
        mq_close(mq);
        return EXIT_FAILURE;
    }

    if (mq_send(mq, "message with prio 9",
                strlen("message with prio 9") + 1, 9) == -1) {
        perror("mq_send prio9");
        mq_close(mq);
        return EXIT_FAILURE;
    }

    if (mq_send(mq, "message with prio 5",
                strlen("message with prio 5") + 1, 5) == -1) {
        perror("mq_send prio5");
        mq_close(mq);
        return EXIT_FAILURE;
    }

    if (mq_close(mq) == -1) {
        perror("mq_close sender");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Run receiver in terminal 1:

```
./07_mqueue_receiver
```

Run sender in terminal 2:

```
./08_mqueue_sender
```

Expected output in terminal 1:

```
receiver <pid>: queue created (/mq_demo_ipc)
receiver <pid>: waiting for sender...
receiver <pid>: got (prio 9): message with prio 9
receiver <pid>: got (prio 5): message with prio 5
receiver <pid>: got (prio 1): message with prio 1
```

Now start two receivers and then one sender:

```
(./07_mqueue_receiver &); (./07_mqueue_receiver &); ./08_mqueue_sender
```

Observe that there is exactly one receiver that consumes all the messages. This is because each receiver starts with `mq_unlink(QUEUE_NAME)`, so they have their own message queue. To which message queue the sender is connected, depends on the moment in time when sender opens the message queue.

Let's remove the `mq_unlink(QUEUE_NAME)` call at the start of the receiver and run the same setup again:

```
(./09_mqueue_receiver &); (./09_mqueue_receiver &); ./08_mqueue_sender
```

Observe that both receivers process messages now and that each message is consumed by exactly one receiver process. Messages are not broadcast to all receivers.

2 Shared Memory

Shared memory is another IPC mechanism. Instead of sending bytes through a kernel buffer, multiple processes map the same memory region into their address spaces. This is often the fastest form of IPC because, after the mapping has been established, the processes access the same bytes directly. The tradeoff is that shared memory only provides shared storage. If multiple processes access the same data concurrently, additional synchronization is required.

Useful manual pages are:

```
man 7 shm_overview
man 3 shm_open
man 2 mmap
man 2 munmap
man 2 ftruncate
```

2.1 A shared mapping between parent and child

The following program creates an anonymous shared mapping using `mmap()`. Because the mapping is created with `MAP_SHARED`, both parent and child see the same integer after `fork()`. The `MAP_ANONYMOUS` flag means that the mapping is not associated with a named object in the file system; it exists only as long as the processes keep it mapped.

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int *counter = mmap(NULL, sizeof(*counter),
                        PROT_READ | PROT_WRITE,
                        MAP_SHARED | MAP_ANONYMOUS,
                        -1, 0);
    if (counter == MAP_FAILED) {
        perror("mmap");
        return EXIT_FAILURE;
    }

    *counter = 0;
```

```c

    pid_t childpid = fork();
    if (childpid == -1) {
        perror("fork");
        munmap(counter, sizeof(*counter));
        return EXIT_FAILURE;
    }

    if (childpid == 0) {
        for (int i = 0; i < 5; i++) {
            (*counter)++;
        }

        printf("child: counter = %d\n", *counter);
        munmap(counter, sizeof(*counter));
        return EXIT_SUCCESS;
    }

    waitpid(childpid, NULL, 0);
    printf("parent: counter = %d\n", *counter);

    munmap(counter, sizeof(*counter));
    return EXIT_SUCCESS;
}
```

Run:

```
./10_shm_fork
```

output:

```
child: counter = 5
parent: counter = 5
```

If `counter` had been a normal variable allocated on the process heap or stack, the child would have modified only its private copy after `fork()`. The parent would still see `0`. Here both processes map the same physical memory pages, so the update becomes visible to the parent as well.

Note that `MAP_SHARED` is essential. With `MAP_PRIVATE`, both processes would again get copy-on-write behavior and modifications would not be shared.

2.2 Named shared memory for independent processes

Anonymous mappings are useful for related processes such as parent and child. For unrelated processes, POSIX shared memory provides a named object that can be opened by different programs. The object is created with `shm_open()`, its size is set with `ftruncate()`, and then it is mapped with `mmap()`.

Writer

TODO: consider calling unlink at writer before opening (or consumer -> look for arguments which variant is the best)

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

#define SHM_NAME "/shm_demo_ipc"

struct shared_data {
    int value;
    char text[64];
};

int main(void) {
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0600);
    if (fd == -1) {
        perror("shm_open writer");
        return EXIT_FAILURE;
    }

    if (ftruncate(fd, sizeof(struct shared_data)) == -1) {
        perror("ftruncate");
        close(fd);
        shm_unlink(SHM_NAME);
        return EXIT_FAILURE;
    }

    struct shared_data *data = mmap(NULL, sizeof(*data),
                                    PROT_READ | PROT_WRITE,
                                    MAP_SHARED, fd, 0);
    if (data == MAP_FAILED) {
        perror("mmap writer");
        close(fd);
        shm_unlink(SHM_NAME);
        return EXIT_FAILURE;
    }

    data->value = 42;
    snprintf(data->text, sizeof(data->text), "hello via shared memory");

    printf("writer: wrote value = %d, text = %s\n",
           data->value, data->text);

    munmap(data, sizeof(*data));
    close(fd);
    return EXIT_SUCCESS;
```

```
}
```

Reader

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

#define SHM_NAME "/shm_demo_ipc"

struct shared_data {
    int value;
    char text[64];
};

int main(void) {
    int fd = shm_open(SHM_NAME, O_RDONLY, 0);
    if (fd == -1) {
        perror("shm_open reader");
        return EXIT_FAILURE;
    }

    struct shared_data *data = mmap(NULL, sizeof(*data),
                                    PROT_READ,
                                    MAP_SHARED, fd, 0);
    if (data == MAP_FAILED) {
        perror("mmap reader");
        close(fd);
        return EXIT_FAILURE;
    }

    printf("reader: read value = %d, text = %s\n",
           data->value, data->text);

    munmap(data, sizeof(*data));
    close(fd);

    if (shm_unlink(SHM_NAME) == -1) {
        perror("shm_unlink");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Run the writer first:

```
./11_shm_writer
```

output:

```
writer: wrote value = 42, text = hello via shared memory
```

Then run the reader:

```
./12_shm_reader
```

output:

```
reader: read value = 42, text = hello via shared memory
```

Important details:

- `shm_open()` returns a file descriptor for the shared memory object. Similar to POSIX message queues, the name must start with `/`, but it is not a normal file system path.
- A newly created shared memory object initially has size `0`. Therefore `ftruncate()` is needed before the object can be mapped with the desired size.
- `mmap(..., MAP_SHARED, ...)` maps the same bytes into the process address space, so writes by one process become visible to another process that maps the same object.
- `shm_unlink(name)` removes the name from the system namespace. As with files, the underlying object is deleted only after the last process has closed and unmapped it.

2.3 Shared memory does not synchronize access

In the examples above, coordination is simple: in the first program the parent waits for the child, and in the second example the writer finishes before the reader starts. In real programs, multiple processes often access the shared region concurrently. Shared memory does not provide mutual exclusion, ordering or notification by itself. If two processes write to the same location at the same time, you have a race condition. Therefore shared memory is commonly combined with semaphores, mutexes in shared memory, or condition variables.